Web Dev with React: useEffect & APIs

Understanding Live vs Static Data, API Waiters, and Automatic Effects

Module: Frontend React Basics **Topic:** Fetching Live Data with useEffect

1. Static vs. Live Data: What's the Difference?

Have you ever noticed how some apps change constantly while others stay fixed?

- **Static Apps:** Show fixed, old data that doesn't change unless someone updates the code. Think of it like looking at a printed photo! A static weather app shows 25°C all day long, regardless of actual weather changes.
- **Live Apps:** Fetch fresh data dynamically from the internet in real-time. Think of it like a live video call! Apps like Swiggy, Zomato, or live weather trackers update status every minute so you see exact real-time information.

📸 Photo vs. Video Call Analogy

Static data is like taking a snapshot picture—it stays frozen in time. Live data is like being on a live video stream—everything is updated second by second!

2. What is an API? Meet the Waiter!

API stands for **Application Programming Interface**—your official data delivery hero!

🍽️ The Restaurant Waiter Analogy

When you sit at a restaurant, you don't go inside the kitchen to cook or grab your dish. You tell the **waiter** what you want. The waiter takes your order to the kitchen (server) and brings back your food (data)!

In web development:

- **You / App:** The customer making a request.
- **API:** The waiter carrying the request back and forth.
- **Server / Database:** The kitchen where all data is stored.

3. The useEffect Hook — The School Bell Alarm!

In React, component functions re-run whenever state updates. But what if we want code to run automatically right when the page loads, without waiting for user button clicks?

That's where `useEffect()` comes in!

> 🔔 **The School Bell Analogy**
>
> `useEffect` works just like a school bell ringing automatically at the start of class! Nobody has to press a button—it triggers on its own as soon as school starts (when the component mounts).

The Empty Dependency Array `[]`:

```
useEffect(() => {
  console.log("Page loaded!");
}, []); // [] = Run ONLY ONCE when page loads!
```

Passing an empty array `[]` as the second argument tells React: *"Run this effect exactly once when the component appears on screen, then stay quiet!"*

4. Code-Along: Building a Daily Motivation Card

Let's combine `fetch()` (calling the waiter) and `useEffect()` (the automatic school bell) to fetch live fun facts and motivation quotes!

STEP-BY-STEP DATA FETCHING PATTERN

1. **Call the Waiter:** Use `fetch('https://api.planrs.in/grade5/fun-facts')` to request live info.
2. **Show Loading State:** Display a spinner or "Loading..." text while waiting for data.
3. **Update UI on Arrival:** Store the incoming data into React state and turn off loading.

```
const [funFact, setFunFact] = useState("");
const [loading, setLoading] = useState(true);

useEffect(() => {
  setLoading(true);
  fetch('https://api.planrs.in/grade5/fun-facts')
    .then(res => res.json())
    .then(data => {
      setFunFact(data.fact);
      setLoading(false);
    });
}, []); // Runs automatically on initial load
```

Practice Questions & Student Worksheet

Test Your Understanding of React useEffect & API Data Fetching

Student Name: _____ **Date:** _____

MULTIPLE CHOICE

Question 1

What is the main difference between static data and live data in web applications?

A. Static data updates automatically every second, while live data never changes.

B. Static data remains fixed like a printed photo, while live data is dynamically fetched fresh like a video call.

C. Static data comes from an API waiter, while live data is hardcoded into HTML.

D. There is no difference between static data and live data.

MULTIPLE CHOICE

Question 2

In the restaurant analogy discussed in class, what role does an API play?

A. The API is the customer ordering food.

B. The API is the chef in the kitchen preparing data.

C. The API is the waiter taking requests to the server and bringing data back to the app.

D. The API is the dining table holding the components.

MULTIPLE CHOICE

Question 3

What happens if you provide an empty dependency array `[]` to a `useEffect()` hook?

A. The effect code runs continuously in an infinite loop.

B. The effect code never runs at all.

C. The effect code runs exactly once when the component first mounts.

D. The effect code runs only when a button is clicked.

MULTIPLE CHOICE

Question 4

What happens if you accidentally forget to include the dependency array in `useEffect(() => { ... })`?

A. The code runs only once when the page loads.

B. The code runs on every single render, potentially causing infinite API calls.

C. React throws a syntax error and stops working.

D. The API waiter disappears from the web page.

SHORT ANSWER

Question 5

Explain in your own words why `useEffect()` is compared to a **School Bell Alarm**.

CODE IDENTIFICATION

Question 6

Look at the code snippet below. Identify the line where the **API Waiter** is called to bring fresh data, and explain what happens when the data arrives:

```
useEffect(() => {
  fetch('https://api.planrs.in/grade5/fun-facts')
    .then(res => res.json())
    .then(data => setFact(data.fact));
}, []);
```

HOMEWORK APPLICATION CHALLENGE

Question 7

Describe how you would use `useEffect()` and `fetch()` to build a **Live Weather Badge** for your favorite city. What steps would your React component follow from page load to displaying the temperature?